



**COMSATS UNIVERSITY ISLAMABAD,
SAHIWAL CAMPUS, COMSATS ROAD OFF
GTROAD, SAHIWAL, PAKISTAN.**

PROJECT REPORT

(Implement an Advance Web -Based Program)

For

**Title: AI Bug Localizer PRO: An Integrated Artificial Intelligence and
Information Security System for Secure Code Analysis**

By:

Nouman Ali	FA24-BCS-288
-------------------	---------------------

INSTRUCTOR:

MAM Rimsha Rafiq

Bachelor of Science in Computer Science (2024-2028) 15,

May 2026

Table of Contents

Summary	4
1. Introduction	4
1.1 Overview	4
1.2 Importance of Secure Software Development	4
1.3 Role of Artificial Intelligence in Software Engineering	5
2. Background and Rationale	5
2.1 Need for Automated Code Analysis	5
2.2 Need for Secure Software Development.....	5
2.3 Need for AI-Assisted Developer Support	5
3. Problem Statement	6
4. Aim and Objectives.....	6
4.1 Aim of the Project	6
4.2 Objectives of the Project	6
5. Scope of the Project	6
5.1 Included Scope	6
5.2 Excluded Scope.....	7
6. Literature and Standards Review	8
6.1 Artificial Intelligence in Software Engineering.....	8
6.2 Static Analysis and Abstract Syntax Trees	8
6.3 AI Risk Management	8
6.4 OWASP Top 10	8
6.5 Secure Software Development Framework	9
6.6 ISO/IEC 27001 and CIA Triad	9
6.7 GDPR, HIPAA, and PCI DSS	9
7. Data Flow Architecture	9
7.1 Code Input Processing	9
7.2 Analysis Processing	9
7.3 Results Processing.....	10
8. Artificial Intelligence Components.....	10
8.1 Intelligent Detection.....	10
8.2 Classification.....	10

8.3 Severity Scoring.....	11
8.4 Ranking.....	11
8.5 Explanation and Recommendation	12
8.6 Human-AI Collaboration	12
9. Information Security Components	14
9.1 Injection Detection.....	14
9.2 Broken Access Control	14
9.3 Cryptographic Failures.....	15
9.4 Security Misconfiguration.....	15
9.5 Vulnerable Dependencies	15
9.6 Authentication and Session Security.....	15
9.7 API Security	16
9.8 Input Validation	16
9.9 Logging and Monitoring	16
9.10 Compliance Awareness.....	16
9.11 Threat Modeling.....	16
10. Methodology	17
11. Security Standards Mapping	18
12. Expected Results and Output	19
13. Evaluation Strategy	20
13.1 Functional Testing.....	20
13.2 Security Test Cases	20
13.3 Accuracy Metrics	20
13.4 Usability Evaluation.....	20
13.5 Performance Evaluation.....	20
14. Strengths of the Project	21
15. Limitations of the Project.....	21
16. Ethical and Legal Considerations	22
17. Future Improvements	22
Conclusion	23
References	24

Summary

AI Bug Localizer PRO is an intelligent secure-code analysis system developed to help developers identify programming bugs, security vulnerabilities, risky coding practices, and compliance-related weaknesses in Python source code. The project combines Artificial Intelligence concepts with Information Security principles to create an automated analysis platform capable of detecting issues, classifying risks, assigning severity scores, ranking findings, and generating recommendations. The system performs static analysis, runtime checks, complexity analysis, vulnerability detection, cryptography review, authentication analysis, API security checks, dependency analysis, configuration analysis, compliance screening, and threat-modeling support. The project demonstrates how AI-based reasoning techniques can improve software quality and support secure software development.

This report explains the project from both Artificial Intelligence and Information Security perspectives. It discusses the background, problem statement, objectives, methodology, conceptual design, AI components, security mechanisms, evaluation methods, ethical considerations, limitations, and future improvements. Recognized standards and frameworks such as OWASP Top 10, NIST SSDF, ISO/IEC 27001, GDPR, HIPAA, and PCI DSS are used to support the discussion. The report concludes that AI Bug Localizer PRO successfully demonstrates how intelligent systems can support cybersecurity and secure software engineering.

1. Introduction

1.1 Overview

Modern software systems must be both functional and secure. A program may execute correctly while still containing vulnerabilities that expose systems to attacks, unauthorized access, or data leakage. As software systems become more complex, manual debugging and manual security analysis become increasingly difficult. AI Bug Localizer PRO was developed to address this challenge by creating an intelligent Python code-analysis platform capable of identifying programming errors and security weaknesses simultaneously. The system accepts Python source code as input and provides structured feedback about vulnerabilities, bugs, complexity, and risk levels. The platform acts as both a development-assistance tool and an educational system that teaches secure coding practices.

1.2 Importance of Secure Software Development

Secure software development ensures that security principles are integrated throughout the software development lifecycle. Developers must consider not only whether code works but also whether it is resistant to misuse, attacks, and exploitation. Common software vulnerabilities such as SQL injection, weak authentication, hardcoded credentials, insecure APIs, and unsafe cryptography can result in financial loss, service disruption, legal penalties, and privacy violations. AI Bug Localizer PRO supports secure development by helping developers identify vulnerabilities during the coding phase instead of after deployment.

1.3 Role of Artificial Intelligence in Software Engineering

Artificial Intelligence is increasingly used in software engineering to automate tasks that traditionally require human expertise. AI systems can assist with bug detection, code recommendation, vulnerability identification, issue prioritization, and automated reasoning. AI Bug Localizer PRO applies rule-based intelligence, classification, and severity scoring, ranking, and recommendation generation to support developer decision-making. Although the system does not rely on neural networks, rule-based expert systems are a recognized branch of Artificial Intelligence because they encode expert knowledge and apply reasoning logic to solve problems.

2. Background and Rationale

2.1 Need for Automated Code Analysis

Code analysis is the process of examining software source code to identify bugs, vulnerabilities, weaknesses, and maintainability problems. Traditional testing methods often identify issues only after execution, whereas static analysis can detect problems before runtime. Manual code review can be time-consuming and inconsistent, especially for large or complex applications. Automated analysis tools improve consistency and help developers identify common problems quickly. AI Bug Localizer PRO combines multiple forms of analysis into one platform, making it easier for developers to detect both software-quality issues and security weaknesses.

2.2 Need for Secure Software Development

Modern applications frequently process sensitive data such as personal information, payment details, and authentication credentials. If security is ignored during development, attackers may exploit weaknesses to compromise systems. NIST's Secure Software Development Framework emphasizes integrating security into every stage of software development. AI Bug Localizer PRO supports this principle by providing automated security checks during development.

The system identifies risks such as:

- SQL injection
- Hardcoded secrets
- Weak cryptographic algorithms
- Missing authentication controls
- Unsafe API usage
- Insecure configurations

This early detection helps reduce the cost and impact of vulnerabilities.

2.3 Need for AI-Assisted Developer Support

Developers often receive large numbers of warnings from analysis tools. Without prioritization, it may be difficult to determine which issues are most dangerous. AI Bug Localizer PRO uses severity scoring and ranking mechanisms to prioritize findings. Critical issues are displayed before lower-risk warnings, helping developers focus on the most important problems first.

This transforms raw detection into actionable decision support.

3. Problem Statement

The main problem addressed by AI Bug Localizer PRO is the lack of simple, intelligent, and security-aware tools for analyzing Python source code. Many developers focus primarily on functionality while ignoring hidden vulnerabilities and insecure coding practices. Existing enterprise-grade tools are often expensive, difficult to use, or unsuitable for educational purposes.

The project addresses this problem by providing a unified platform capable of:

1. Detecting programming bugs and syntax errors.
2. Identifying runtime and logical problems.
3. Detecting security vulnerabilities.
4. Reviewing cryptographic and authentication practices.
5. Performing compliance-awareness checks.
6. Ranking issues according to severity.
7. Providing explanations and remediation guidance.
8. Presenting results through a readable dashboard.

4. Aim and Objectives

4.1 Aim of the Project

The aim of the project is to develop an AI-assisted secure code analysis system that improves software quality, supports secure programming, and helps developers understand vulnerabilities in Python source code.

4.2 Objectives of the Project

The objectives of the project are:

- To build an automated code-analysis pipeline.
- To perform static analysis on Python source code.
- To identify runtime-related issues.
- To calculate code complexity and risk indicators.
- To apply AI-based rule reasoning for issue detection.
- To classify and prioritize detected vulnerabilities.
- To detect common information-security weaknesses.
- To provide remediation recommendations.
- To map findings to security standards.
- To present results through an interactive dashboard.
- To support both education and secure software development.

5. Scope of the Project

5.1 Included Scope

The project focuses on Python source-code analysis and includes:

- Static code analysis
- Runtime error detection
- Complexity analysis
- Security vulnerability detection
- Cryptography review
- Authentication and authorization review
- API security analysis
- Dependency analysis
- Input validation review
- Compliance awareness checks
- Threat-modeling indicators
- Dashboard and reporting support

5.2 Excluded Scope

The project does not include:

- Multi-language support
- Legal compliance certification
- Enterprise penetration testing
- Full dynamic application security testing
- Complete sandboxing infrastructure

Project scope

✓ Included scope	✗ Excluded scope
Static code analysis	Multi-language support
Runtime error detection	Legal compliance certification
Security vulnerability detection	Enterprise penetration testing
Cryptography and auth review	Full dynamic SAST
Compliance awareness checks	Full sandboxing infrastructure
Dashboard and reporting	Enterprise vulnerability-DB integration

- Enterprise vulnerability-database integration

6. Literature and Standards Review

6.1 Artificial Intelligence in Software Engineering

Artificial Intelligence techniques are widely used in software engineering for automating complex tasks such as bug detection, code recommendation, and vulnerability analysis. AI Bug Localizer PRO uses rule-based reasoning to simulate expert decision-making. The system applies predefined rules to identify insecure coding patterns and software weaknesses. This approach improves explainability because users can understand why a warning was generated.

6.2 Static Analysis and Abstract Syntax Trees

Static analysis inspects source code without executing it. Python's Abstract Syntax Tree (AST) module allows applications to analyze source-code structure programmatically.

The project uses AST analysis to inspect:

- Function calls
- Variable usage
- Imports
- Assignments
- Syntax structure

AST analysis provides more meaningful insights than simple keyword searching.

6.3 AI Risk Management

AI systems can produce false positives and false negatives. NIST's AI Risk Management Framework emphasizes transparency, accountability, and trustworthy AI systems.

AI Bug Localizer PRO reduces AI-related risks by:

- Explaining findings clearly
- Assigning severity levels
- Encouraging human review
- Presenting results as advisory rather than authoritative

6.4 OWASP Top 10

The OWASP Top 10 identifies critical web application security risks such as:

- Broken access control
- Cryptographic failures
- Injection attacks
- Security misconfiguration
- Vulnerable components

- Authentication failures

AI Bug Localizer PRO incorporates these concepts into its security analyzers.

6.5 Secure Software Development Framework

NIST SP 800-218 promotes secure software development practices integrated throughout the software development lifecycle. The project supports these principles by helping developers identify vulnerabilities early during development.

6.6 ISO/IEC 27001 and CIA Triad

ISO/IEC 27001 focuses on confidentiality, integrity, and availability.

The project supports:

- Confidentiality through secret detection and encryption checks
- Integrity through injection detection and secure validation
- Availability through secure configuration and monitoring awareness

6.7 GDPR, HIPAA, and PCI DSS

The system includes compliance-awareness checks related to:

- GDPR for personal-data protection
- HIPAA for healthcare information protection
- PCI DSS for payment-card security

These checks are advisory and educational rather than legally certifying compliance.

7. Data Flow Architecture

7.1 Code Input Processing

1. User Input → Code Editor (Streamlit ACE)
2. Code Stored → In-memory storage
3. Preprocessing → Syntax validation
4. AST Generation → Abstract Syntax Tree creation
5. Tokenization → Code token extraction

7.2 Analysis Processing

1. Static Analysis → Pattern matching on AST
2. Runtime Analysis → Execute code with error handling
3. Security Modules → Run all security checks in parallel
4. Aggregation → Combine all findings
5. De-duplication → Remove redundant issues

7.3 Results Processing

- 1. Issue Ranking → ML model scores and ranks issues
- 2. Severity Classification → HIGH, MEDIUM, LOW, CRITICAL
- 3. Fix Generation → Automated fix suggestions
- 4. Explanation Generation → AI-powered descriptions
- 5. Report Creation → Dashboard rendering & PDF export

8. Artificial Intelligence Components

8.1 Intelligent Detection

The system uses intelligent rule-based detection mechanisms to identify risky coding patterns.

Examples include:

- SQL query concatenation
- Hardcoded passwords
- Unsafe shell execution
- Weak cryptographic algorithms
- Missing input validation

These rules represent encoded expert knowledge.

8.2 Classification

The system classifies findings into categories such as:

Table 1: Common Software Errors and Security Vulnerabilities Identified by the System

Issue Type	Description
Syntax error	Mistakes in code structure that prevent compilation or execution
Runtime error	Errors that occur while the program is running
SQL injection	Malicious SQL code inserted to manipulate database queries
Weak hashing	Use of insecure hashing algorithms (e.g., MD5, SHA1)
Hardcoded secret	Sensitive data (passwords, API keys) written directly in code
Authentication weakness	Flaws in login or identity verification mechanisms
Configuration issue	Misconfigured settings that expose systems or reduce security
Compliance warning	Violations of security standards, policies, or regulations

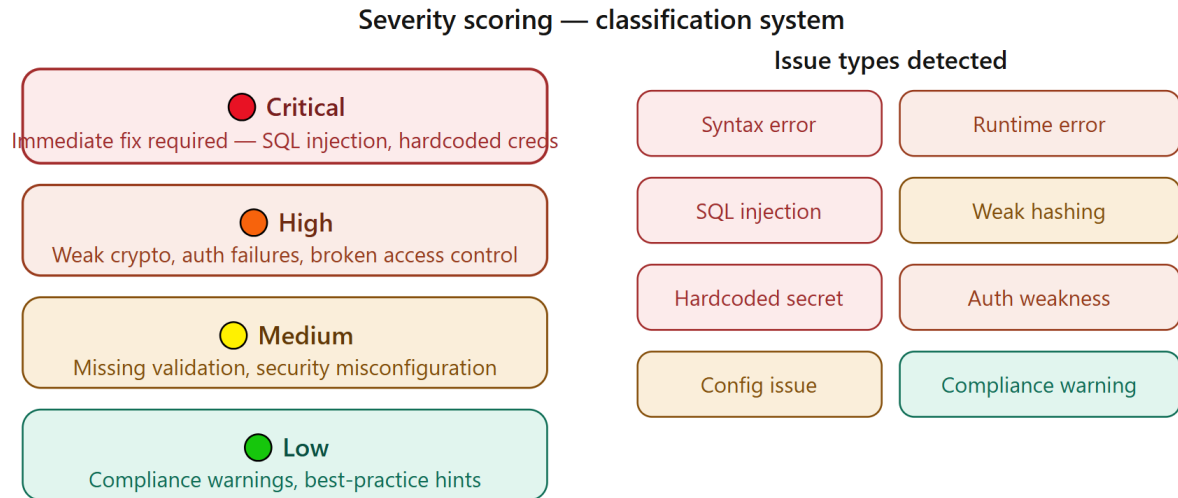
Classification improves readability and usability.

8.3 Severity Scoring

Each issue receives a severity score based on potential impact and urgency.

Severity levels include:

- Low
- Medium
- High
- Critical

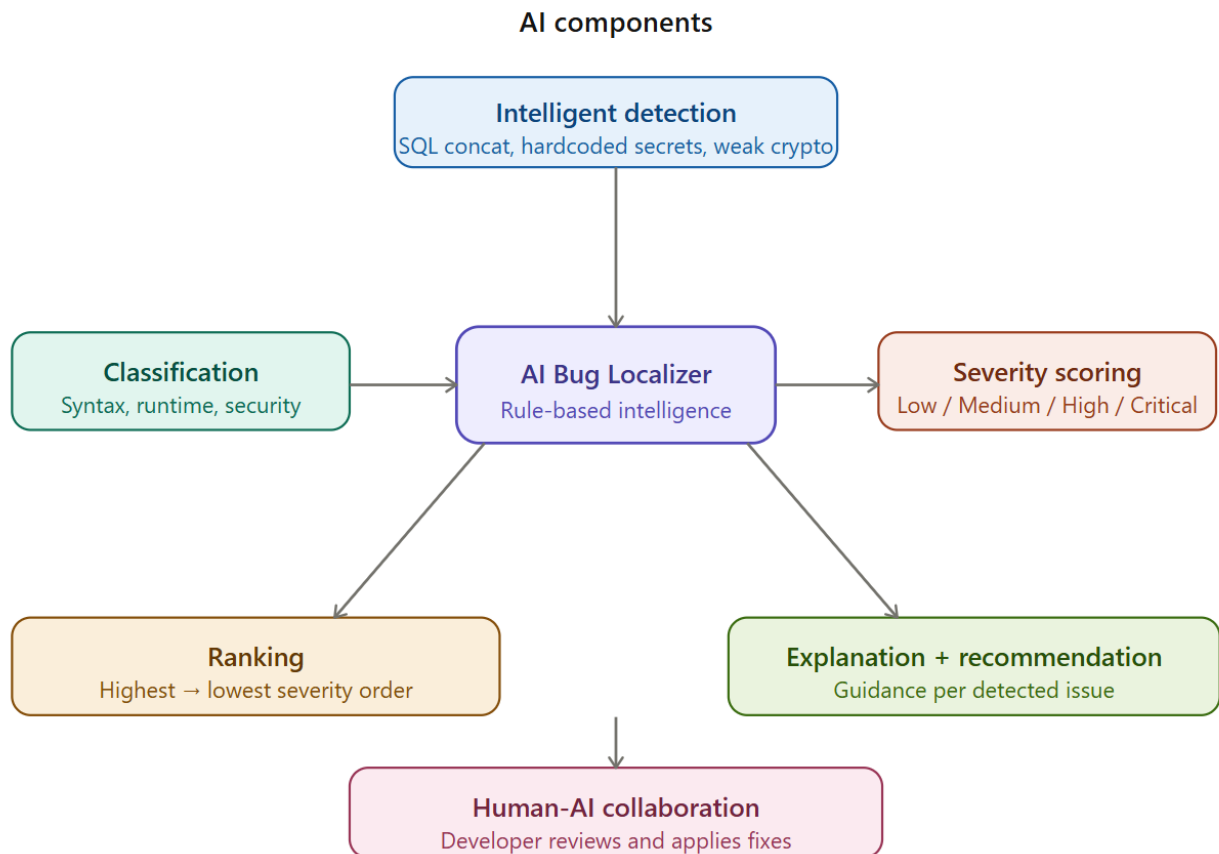


Scoring helps developers prioritize fixes.

8.4 Ranking

After scoring, findings are ranked from highest to lowest severity.

This prevents users from being overwhelmed by large numbers of warnings.



8.5 Explanation and Recommendation

The system provides explanations and remediation guidance for detected issues.

For example:

- Hardcoded secrets → Use environment variables.
- Weak hashing → Use bcrypt or Argon2.
- SQL concatenation → Use parameterized queries.

This educational approach improves secure coding awareness.

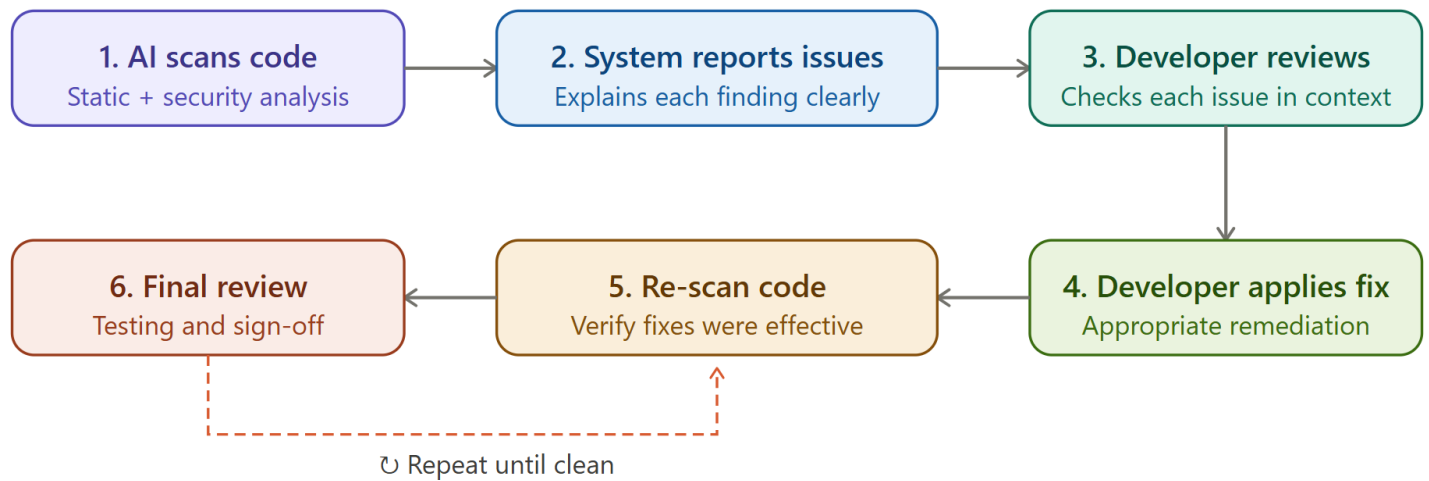
8.6 Human-AI Collaboration

The system should be seen as a collaborative assistant, not an authority that replaces humans. The AI helps identify and prioritize issues, but the human developer must review the results. This is especially important because code context matters. A warning may be valid in production code but acceptable in a controlled test example.

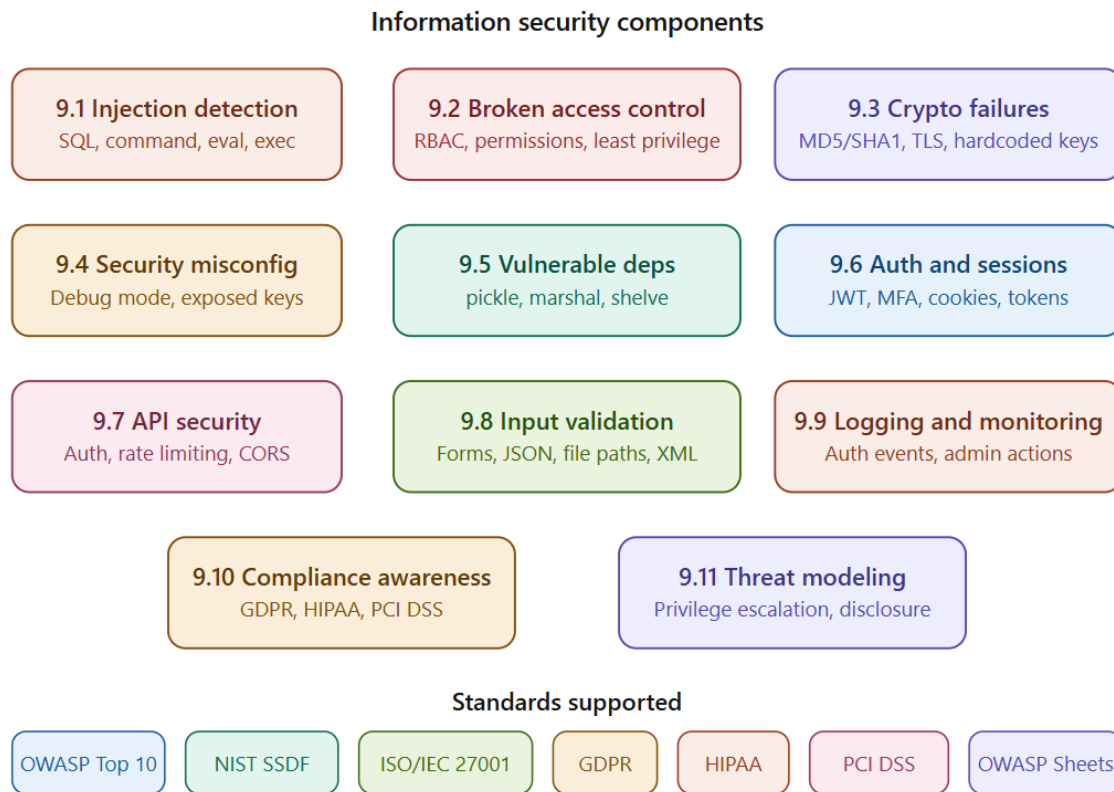
The ideal workflow is:

1. The AI system scans the code.
2. The system reports and explains issues.
3. The developer reviews each issue.
4. The developer applies appropriate fixes.
5. The code is scanned again.
6. Final review and testing are performed.

Human-AI collaboration workflow



9. Information Security Components



9.1 Injection Detection

The system detects unsafe patterns related to:

- SQL injection
- Command injection
- Unsafe eval() usage
- Unsafe exec() usage

Recommended defenses include parameterized queries and strict input validation.

9.2 Broken Access Control

The project checks for missing authorization logic and insecure direct object references.

Recommended defenses include:

- Role-based access control
- Permission checks
- Ownership validation

- Least privilege

9.3 Cryptographic Failures

The system identifies:

- MD5 and SHA1 usage
- Hardcoded cryptographic keys
- Disabled TLS verification
- Static initialization vectors

Recommended solutions include modern encryption and secure password hashing.

9.4 Security Misconfiguration

The project checks for:

- Debug mode enabled
- Exposed secret keys
- Insecure SSL configuration
- Unsafe deployment settings

Secure defaults and configuration management are recommended.

9.5 Vulnerable Dependencies

The project identifies risky imports and unsafe deserialization modules such as:

- pickle
- marshal
- shelve

Safer alternatives and dependency management are recommended.

9.6 Authentication and Session Security

The system checks for:

- Weak passwords
- Missing token expiry
- Weak JWT secrets
- Missing MFA
- Insecure cookies

Recommended solutions include strong authentication mechanisms and secure session management.

9.7 API Security

The system checks for:

- Missing authentication
- Missing rate limiting
- Hardcoded API keys
- Insecure CORS policies
- Missing logging

These checks support secure API development.

9.8 Input Validation

The project identifies missing validation in:

- Forms
- Request parameters
- JSON input
- File paths
- XML parsing

Allow lists and schema validation are recommended.

9.9 Logging and Monitoring

Security logging and monitoring help detect attacks, investigate incidents, and maintain accountability. Without logs, an organization may not know that an attack happened or may be unable to determine what was affected. The project checks for missing API logging and insufficient security monitoring patterns. The recommended defense is to log authentication events, authorization failures, sensitive operations, administrative actions, and suspicious activity. Logs should avoid exposing secrets and should be protected from tampering.

9.10 Compliance Awareness

The project identifies indicators related to:

- Personal data exposure
- Health information exposure
- Payment-card handling risks

These checks support compliance awareness.

9.11 Threat Modeling

Threat-modeling checks include:

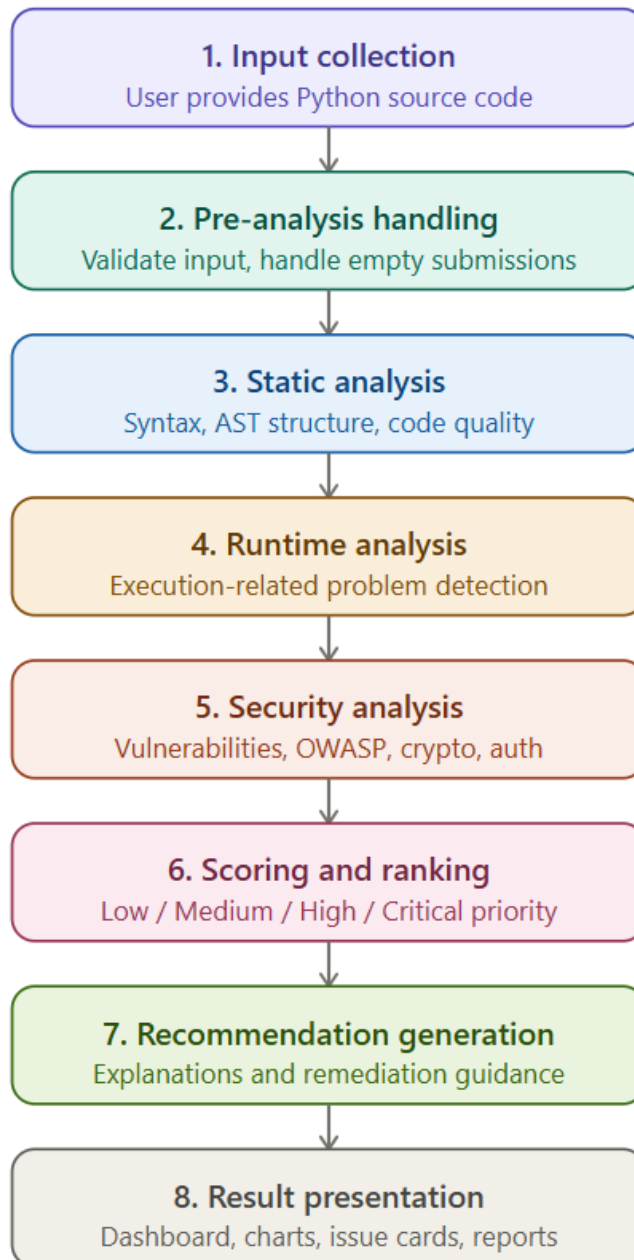
- Privilege escalation risks

- Data exposure risks
- Missing authorization
- Information disclosure

This encourages proactive security thinking.

10. Methodology

Analysis pipeline — AI Bug Localizer PRO



Frontend	Streamlit, Streamlit ACE	Web UI framework
Visualization	Plotly, Pandas	Charts and data visualization
Code Analysis	AST, Regex, Pattern Matching	Static code analysis
Security	cryptography, hashlib	Cryptographic operations
ML/AI	sklearn (implied), Custom ML	Issue ranking & classification
Report Generation	Report Lab, python-docks	PDF and Document generation
AI Chat	LLM Integration (Custom)	Natural language processing
Backend	Python 3.x	Core logic implementation

Table 2: Technology Stack

11. Security Standards Mapping

Table 3: Security Standards and Frameworks Supported by the Project

Standard / Framework	Relevant Concept	Project Support
OWASP Top 10	Application Security Risks	Injection detection, authentication checks, configuration review
NIST SSDF	Secure Development	Early vulnerability detection
ISO/IEC 27001	CIA Triad and Risk Management	Encryption checks, logging, secret detection
GDPR	Personal Data Protection	Data-exposure awareness
HIPAA	Health Information Protection	PHI awareness and logging
PCI DSS	Payment Security	Card-data detection and encryption awareness

OWASP Cheat Sheets	Secure Coding Guidance	Password-storage and cryptography recommendations
--------------------	------------------------	---

12. Expected Results and Output

The system is expected to generate:

- Detected issue lists
- Severity scores
- Severity labels
- Vulnerability categories
- Descriptions of findings
- Remediation recommendations
- Dashboard metrics
- Security-focused summaries
- Optional report generation

Example outputs include:

Table 4: Example Security Findings, Risks, and Recommended Actions

Example Finding	Security Risk	Recommended Action
Hardcoded Password Warning	Exposure of sensitive credentials	Store secrets in environment variables or secure vaults
Weak Cryptography Alert	Use of insecure hashing or encryption algorithms	Replace with bcrypt, Argon2, or modern cryptographic standards
Missing Token Expiry Detection	Increased risk of session hijacking or token misuse	Add secure expiration (<code>exp</code>) claims to tokens
SQL Injection Warning	Unauthorized database access or manipulation	Use parameterized queries and input validation
Insecure HTTP Usage Alert	Data transmission without encryption	Replace HTTP with HTTPS and enforce TLS security

13. Evaluation Strategy

13.1 Functional Testing

Functional testing verifies that all features work correctly.

Test cases include:

- Syntax errors
- SQL injection patterns
- Hardcoded secrets
- Missing validation
- Weak hashing

13.2 Security Test Cases

Security testing ensures analyzers correctly identify vulnerabilities while avoiding excessive false positives.

13.3 Accuracy Metrics

Evaluation metrics include:

- True positives
- False positives
- True negatives
- False negatives
- Precision
- Recall

13.4 Usability Evaluation

Usability testing measures:

- Ease of use
- Clarity of recommendations
- Dashboard readability
- Navigation simplicity

13.5 Performance Evaluation

Performance testing measures analysis speed for small, medium, and large source-code samples.

14. Strengths of the Project

Table 5: Key Strengths and Benefits of the Proposed System

Strength	Description	Benefit
Integration of AI and Cybersecurity	Combines intelligent code analysis with security vulnerability detection	Provides a unified platform for software quality and security assessment
Educational Explanations and Recommendations	Explains detected issues and suggests remediation steps	Helps students and developers learn secure coding practices
Broad Vulnerability Coverage	Detects issues related to OWASP risks, cryptography, authentication, APIs, and compliance	Improves overall security awareness and analysis depth
Severity Prioritization	Assigns severity scores and ranks findings according to risk	Helps users focus on the most critical vulnerabilities first
Interactive Dashboard Design	Presents results using categorized views, metrics, and visual elements	Improves readability and user experience
Extensible Modular Architecture	Allows additional analyzers and security checks to be added easily	Supports future scalability and customization
Practical Support for Secure Coding	Provides actionable feedback during software development	Encourages secure software development practices early in the SDLC

15. Limitations of the Project

The project also has limitations:

- Rule-based detection may produce false positives.
- Context understanding is limited.
- Runtime execution may require secure sandboxing.
- Compliance checks are advisory only.
- Dependency analysis is simplified.
- Business-context risk analysis is limited.

These limitations provide opportunities for future improvements.

16. Ethical and Legal Considerations

The project should be used responsibly.

Important considerations include:

- Protecting uploaded source code
- Avoiding unauthorized security testing
- Preventing misuse of scanning features
- Clarifying that results are advisory
- Maintaining transparency in AI reasoning

17. Future Improvements

Table 6: Proposed Future Enhancements and Their Benefits

Improvement	Description	Expected Benefit
Advanced AST Analysis	Improve Abstract Syntax Tree processing with deeper structural understanding	More accurate code analysis and vulnerability detection
Taint Analysis	Track untrusted input through the application flow	Better detection of injection and input-based vulnerabilities
Data-Flow Analysis	Analyze how data moves between variables and functions	Improved detection of insecure data handling
Control-Flow Analysis	Examine execution paths and logical branches	Better understanding of runtime behavior and hidden risks
CWE and CVE Mapping	Map findings to Common Weakness Enumeration and Common Vulnerabilities and Exposures databases	Alignment with industry-standard vulnerability classification
Dependency-Vulnerability Integration	Connect dependency analysis with vulnerability databases	Detection of outdated or vulnerable third-party libraries
Sandboxed Execution	Execute code in isolated environments with restricted permissions	Safer runtime analysis of untrusted code
Multi-File Project Analysis	Support scanning across complete software projects	More comprehensive and realistic analysis
SARIF Export Support	Export results using the Static Analysis Results Interchange Format	Easier integration with security and developer tools
CI/CD Integration	Integrate scanning into automated development pipelines	Continuous security testing during development
Confidence Scoring	Assign confidence levels to detected findings	Better prioritization and reduced uncertainty

Secure Coding Tutorials	Provide educational guidance linked to findings	Improved learning experience for developers and students
Machine-Learning Vulnerability Models	Use trained ML models for advanced vulnerability detection	Smarter and adaptive analysis capabilities
Role-Based Reporting	Generate different reports for students, developers, and managers	Improved usability for different audiences
Improved False-Positive Suppression	Reduce incorrect warnings through smarter filtering	Higher accuracy and better user trust

Conclusion

AI Bug Localizer PRO is a comprehensive academic project that successfully combines Artificial Intelligence and Information Security concepts.

The system demonstrates AI capabilities through:

- Automated reasoning
- Static and runtime analysis
- Classification
- Severity scoring
- Ranking
- Recommendation generation
- Human-AI collaboration

The project demonstrates Information Security principles through:

- Vulnerability detection
- OWASP alignment
- Cryptography review
- Authentication analysis
- API security
- Compliance awareness
- Threat modeling

The project addresses a real-world challenge by helping developers identify software bugs and security weaknesses early in development.

Although the system has limitations, it provides a strong educational and technical foundation for future development.

AI Bug Localizer PRO demonstrates how intelligent systems can improve secure software engineering and cybersecurity awareness.

References

- European Parliament & Council of the European Union. (2016). *General Data Protection Regulation (GDPR)*. <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>
- International Organization for Standardization. (2022). *ISO/IEC 27001:2022 Information Security Management Systems*. <https://www.iso.org/standard/27001>
- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/2021/>
- OWASP Foundation. (n.d.). *Cryptographic Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- OWASP Foundation. (n.d.). *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- PCI Security Standards Council. (2022). *PCI DSS v4.0*. <https://www.pcisecuritystandards.org/>
- Python Software Foundation. (2026). *ast — Abstract Syntax Trees*. <https://docs.python.org/3/library/ast.html>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure Software Development Framework (NIST SP 800-218)*. National Institute of Standards and Technology.
- Tabassi, E. (2023). *Artificial Intelligence Risk Management Framework (NIST AI RMF 1.0)*. National Institute of Standards and Technology.
- U.S. Department of Health & Human Services. (n.d.). *HIPAA Security Rule Guidance Material*. <https://www.hhs.gov/hipaa/>